

MDS-008 — Component Library

External publication draft

MDS-008 — Component Library

Components do not decide. They faithfully render the decisions already made.

Purpose

Every previous specification has progressively transformed behavioural understanding into presentation.

The MDL established:

- Vision
- Principles
- Mental Model
- Interaction
- Composition

The MDS established:

- Design Tokens
- Colour
- Materials
- Typography
- Motion
- Composition Engine
- Tile Framework

MDS-008 defines the final architectural layer.

The Component Library is responsible for physically implementing the behavioural presentation described by resolved Tiles.

Unlike traditional UI frameworks, Components possess almost no behavioural responsibility.

By the time a Component is created:

- Behaviour has been solved.
- Hierarchy has been solved.
- Materials have been solved.
- Typography has been solved.
- Motion has been solved.
- Interaction has been solved.
- Tiles have been solved.

Components simply render.

Relationship to Previous Specifications

[text]

Vision

↓

Principles

↓

Mental Model

↓

Interaction

↓

Composition

↓

Tokens

↓

Colour

↓

Materials

↓

Typography

↓

Motion

↓

Composition Engine

↓

Tile Framework

↓

Component Library

↓

Rendering

The Component Library consumes:

- Resolved Tiles
- Material Profiles
- Typography Profiles
- Motion Profiles
- Interaction Profiles

It produces:

- Platform Components
- Render Trees
- Accessible UI

Scope

This specification defines:

- Component Philosophy
- Component Taxonomy
- Component Contracts
- Component Lifecycle
- Component Composition
- Rendering Architecture
- Platform Components
- Accessibility Contracts
- Runtime Rendering
- Component Optimisation

This specification intentionally does **not** define:

- Behaviour
- Hierarchy
- Runtime World
- Expressions
- Tiles
- Design Tokens

Those systems already solved the problem.

The Component Library simply implements their decisions.

Core Question

MDS-008 exists to answer one question.

How should resolved Tiles become concrete user interface implementations?

Not:

How should the application behave?

Component Statement

Within Mosaic:

Components render. They never reason.

This is the single most important architectural principle of the Component Library.

Component Responsibilities

The Component Library separates implementation into several conceptual layers.

[text]

Resolved Tile

↓

Component Contract

↓

Platform Component

↓

Rendering

↓

Pixels

Each layer contributes exactly one responsibility.

Expected Outcome

After reading MDS-008 contributors should understand:

- why Components intentionally remain simple,
- how Component Contracts work,
- how Components remain platform independent,
- how rendering stays replaceable,
- how accessibility integrates,
- how runtime presentation becomes visible,

without redefining behaviour or runtime architecture.

Repository Structure

[text]
design/

■■■ mds/

■■■ MDS-008 Component Library/

README.md

00-document-control.md

01-component-philosophy.md

02-component-taxonomy.md

03-component-contracts.md

04-component-lifecycle.md

05-component-composition.md

06-rendering-architecture.md

07-platform-components.md

08-accessibility-contracts.md

09-runtime-rendering.md

10-component-optimisation.md

11-governance.md

12-adrs.md

13-contributor-guidance.md

glossary.md

references.md

Dependencies

Required reading:

- MDL-001 → MDL-005
- MDS-001 → MDS-007

Downstream specifications:

There are currently no downstream Design System specifications.

Platform implementation guides may build upon this specification.

Review Status

Status

Draft

Owner

Lead UI Platform Architecture Team

Next File

00-document-control.md

Document Control

Document Information

Property	Value
Document ID	MDS-008
Title	Mosaic Design System — Component Library
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead UI Platform Architecture Team
Parent Specifications	MDL-001 → MDL-005, MDS-001 → MDS-007
Repository	/design/mds/MDS-008 Component Library/

Purpose

MDS-008 defines the Component Library used throughout Mosaic.

The Component Library is the final implementation layer of the entire Mosaic Design Language.

Everything before this specification determines:

- behaviour,
- hierarchy,
- materials,
- typography,
- motion,
- presentation.

The Component Library determines only:

How those solved decisions become visible.

Unlike conventional component libraries, Mosaic Components intentionally possess almost no behavioural responsibility.

Components are implementation.

Not architecture.

Authority

MDS-008 governs:

- Component philosophy

- Component taxonomy
- Component contracts
- Component lifecycle
- Component composition
- Rendering architecture
- Platform components
- Accessibility contracts
- Runtime rendering
- Component optimisation

This specification intentionally does **not** govern:

- Behaviour
- Runtime World
- Composition
- Expressions
- Tiles
- Design Tokens

Those systems already solved runtime understanding.

Components simply render it.

Relationship To MDS

The Component Library sits at the very end of the runtime architecture.

[mermaid]
flowchart TD

```

Vision
Vision --> Principles
Principles --> MentalModel
MentalModel --> Interaction
Interaction --> Composition
Composition --> Tokens
Tokens --> Colour
Colour --> Materials
Materials --> Typography
Typography --> Motion
Motion --> CompositionEngine
CompositionEngine --> TileFramework
TileFramework --> ComponentLibrary
ComponentLibrary --> Rendering
Rendering --> User

```

Every previous specification influences Components.

Components influence nothing upstream.

Design Intent

Traditional UI frameworks frequently behave like this.

[text]
Components

↓

State

↓

Behaviour

↓

Rendering

Mosaic intentionally reverses this dependency.

[text]
Behaviour

↓

Composition

↓

Tiles

↓

Components

↓

Rendering

Components become passive renderers of behavioural presentation.

Reader Expectations

Before reading this specification contributors should already understand:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 → MDS-007

MDS-008 assumes every architectural decision has already been made.

Its responsibility is implementation.

Architectural Scope

The Component Library defines:

- component contracts
- rendering behaviour
- platform implementation
- accessibility implementation
- runtime rendering
- component optimisation

It intentionally avoids implementation-specific frameworks becoming architectural concepts.

Flutter, React, SwiftUI and Compose remain implementation targets.

Not design abstractions.

Stability

Expected lifetime.

Artefact	Expected Lifetime
Platform Widgets	Months
Rendering APIs	Months
Framework Integrations	Years
Component Contracts	Years
Component Philosophy	Decades

Frameworks evolve.

Component philosophy should remain recognisably Mosaic.

Success Criteria

MDS-008 succeeds when:

- Components remain behaviourally simple
- platform implementations remain replaceable
- accessibility is automatically inherited
- rendering remains deterministic
- contributors naturally think in Tiles rather than Components
- the runtime architecture remains invisible beneath implementation

Users should never perceive Components.

They should simply experience a Companion that always behaves consistently.

Review Status

Status

Draft

Dependencies

- MDL-001 → MDL-005
- MDS-001 → MDS-007

Supersedes

None.

Next File

01-component-philosophy.md

Component Philosophy

Purpose

Before defining Component Contracts, platform implementations or rendering behaviour, contributors must first understand what a Component represents within Mosaic.

Many UI frameworks place Components at the centre of application architecture.

Components:

- manage state,
- own behaviour,
- control interaction,
- perform rendering.

Mosaic intentionally rejects this model.

By the time a Component exists...

Every important architectural decision has already been made.

Components exist for one purpose.

To faithfully render the user's World.

Philosophy Statement

Components implement presentation. They never determine understanding.

Everything within the Component Library derives from this statement.

Why Components Exist

The Composition Engine produces:

- Expressions

The Tile Framework produces:

- Tiles

The Runtime Tile Resolver produces:

- fully resolved presentation.

Rendering technologies cannot consume these architectural concepts directly.

Components provide the final bridge.

[text]

Resolved Tile

↓

Component

↓

Rendering

↓

User

Components are implementation.

Nothing more.

Components Are Passive

A Component should never ask:

"What should happen?"

Instead it should ask:

"How should I render what has already been decided?"

This distinction dramatically simplifies application architecture.

Behaviour belongs elsewhere.

Components Never Solve Behaviour

Incorrect.

[text]
Button

↓

Play

↓

Behaviour

Correct.

[text]
Action Tile

↓

Resolved Interaction

↓

Action Component

↓

Rendering

The behavioural decision already exists.

The Component merely implements it.

Components Consume Contracts

Components should consume only resolved contracts.

Examples include:

- Material Profile
- Typography Profile
- Motion Profile
- Interaction Profile
- Accessibility Profile

Components should never independently resolve:

- hierarchy
- Materials
- Typography
- Motion

The runtime already solved those decisions.

Components Are Replaceable

A Hero Component may be implemented using:

- Flutter
- React
- SwiftUI
- Compose
- future frameworks

Every implementation should communicate the same Tile.

Frameworks are temporary.

Component behaviour should remain stable.

Components Preserve Identity

A Component should faithfully preserve Tile identity.

Example.

[text]
Hero Tile

↓

Hero Component

Not.

[text]
Hero Tile

↓

Large Card

Component naming should reflect architectural identity.

Not visual appearance.

Components Respect Materials

Components receive Material behaviour.

Examples.

Hero Component.

↓

Hero Material.

Metadata Component.

↓

Surface Material.

Overlay Component.

↓

Overlay Material.

Components should never invent Material behaviour independently.

Components Respect Typography

Typography is resolved before Components exist.

Examples.

Heading.

↓

Heading Component.

Supporting.

↓

Supporting Component.

Caption.

↓

Caption Component.

Editorial hierarchy should remain entirely runtime driven.

Components Respect Motion

Motion is likewise inherited.

Hero Component.



Hero Motion.

Timeline Component.



Supporting Motion.

Overlay Component.



Overlay Motion.

Components execute motion.

They never define it.

Components Respect Interaction

Interaction belongs to behavioural intent.

Components implement:

- pointer events,
- touch events,
- keyboard events,
- accessibility actions.

They should never redefine behavioural outcomes.

Components Respect Accessibility

Accessibility is automatic.

Components receive:

- accessible typography,
- accessible Materials,
- accessible Motion,
- accessible interaction.

They should not independently invent accessibility behaviour.

Accessibility belongs to the runtime architecture.

Components Are Stateless

Whenever practical...

Components should remain stateless.

State belongs to:

- Runtime World,
- Composition Engine,
- Tile Framework.

Components render current presentation.

Nothing more.

Components Are Predictable

Given identical Resolved Tiles...

Components should always produce identical presentation.

Deterministic rendering simplifies:

- testing,
- replay,
- debugging,
- optimisation.

Predictability remains more valuable than clever implementation.

Components Are Disposable

Unlike Tiles...

Components are disposable.

They may be:

- recreated,
- recycled,
- virtualised,
- pooled.

Behavioural identity survives because:

Tiles survive.

Not because Components survive.

This separation significantly improves implementation flexibility.

Components Across Platforms

Every platform should implement the same Component contracts.

Flutter.

↓

Hero Component.

React.

↓

Hero Component.

SwiftUI.

↓

Hero Component.

Compose.

↓

Hero Component.

Implementations differ.

Architectural identity remains unchanged.

Plugins

Extensions never provide Components.

Plugins contribute:

- behaviour,
- Expressions,
- information.

The Tile Framework resolves presentation.

Platform implementations provide Components.

This guarantees one coherent implementation model.

Good Examples

Playback

Resolved Hero Tile.



Hero Component.



Rendering.

Behaviour remains unchanged.

Reading

Resolved Timeline Tile.



Timeline Component.



Rendering.

Editorial understanding remains intact.

Music

Resolved Relationship Tile.



Relationship Component.



Presentation.

Components remain implementation only.

Anti-patterns

Smart Components

Components solving runtime behaviour.

Stateful Components

Components becoming behavioural authorities.

Platform Behaviour

Different frameworks producing different behavioural outcomes.

Component Styling

Components independently inventing Material or Typography behaviour.

Component Philosophy Model

[mermaid]
flowchart TD

```
ResolvedTile  
ResolvedTile --> ComponentContract  
ComponentContract --> PlatformComponent  
PlatformComponent --> Rendering  
Rendering --> User
```

Components faithfully implement runtime presentation.

Nothing upstream depends upon them.

Relationship To Future Chapters

The remaining chapters define how this philosophy becomes implementation.

Including:

- Component Taxonomy
- Component Contracts
- Component Lifecycle
- Component Composition
- Rendering Architecture
- Platform Components
- Accessibility Contracts
- Runtime Rendering

Every implementation decision should reinforce the philosophy established here.

Summary

Components are the thinnest architectural layer in Mosaic.

They should:

- render,
- adapt,
- implement,
- disappear.

They should never:

- solve,
- decide,
- infer,
- reinterpret.

By the time a Component exists, the user's World has already been completely understood.

The Component simply makes that understanding visible.

Review Status

Status

Draft

Next File

02-component-taxonomy.md

Component Taxonomy

Purpose

The Tile Framework defines the behavioural vocabulary of Mosaic.

The Component Library defines the implementation vocabulary.

Unlike Tiles, which represent behavioural presentation, Components represent concrete rendering primitives.

Component Taxonomy exists to ensure every implementation remains:

- predictable,
- reusable,
- platform independent,
- behaviourally passive.

The taxonomy should remain intentionally small.

Components should solve rendering.

Nothing more.

Definition

Within MDS, **Component Taxonomy** is defined as:

The canonical classification of implementation primitives responsible for rendering resolved Tiles.

Components communicate implementation.

They intentionally do **not** communicate behaviour.

Why A Taxonomy Exists

Without a shared Component vocabulary, implementations naturally diverge.

Examples.

- PosterCard
- MediaCard
- HeroWidget
- EpisodePanel
- AlbumView

These names describe application features.

Not implementation.

Instead Mosaic defines a stable implementation vocabulary independent from runtime behaviour.

Component Families

The Component Library defines the following conceptual component families.

[text]

Container

↓

Content

↓

Media

↓

Text

↓

Action

↓

Indicator

↓

Navigation

↓

Overlay

↓

Input

↓

Utility

Every Component belongs to exactly one primary family.

Container Components

Purpose.

Provide structural implementation.

Examples include:

- panels
- regions
- stacks
- groups
- layouts

Container Components never determine hierarchy.

They simply implement the structure already resolved by the Tile Framework.

Content Components

Purpose.

Render resolved content.

Examples include:

- content bodies
- summaries
- descriptions
- metadata collections

Content Components communicate information.

They never determine its importance.

Media Components

Purpose.

Render media.

Examples include:

- artwork
- posters
- thumbnails
- backdrops
- video surfaces

Media Components implement presentation.

They never determine Material behaviour.

Text Components

Purpose.

Render resolved typography.

Examples include:

- headings
- body text
- captions
- labels

Text Components consume resolved Typography Profiles.

They never select typography independently.

Action Components

Purpose.

Implement behavioural interaction.

Examples include:

- buttons
- segmented controls
- icon actions
- contextual actions

Action Components render Interaction Profiles.

They never define interaction behaviour.

Indicator Components

Purpose.

Communicate runtime state.

Examples include:

- progress
- badges
- status
- loading
- activity

Indicators render resolved behavioural state.

They never infer it.

Navigation Components

Purpose.

Implement navigation affordances.

Examples include:

- navigation bars
- side rails
- tab bars

- breadcrumbs

Navigation Components remain behaviourally passive.

The Runtime World determines navigation state.

Overlay Components

Purpose.

Render temporary presentation.

Examples include:

- dialogs
- sheets
- menus
- playback controls

Overlay Components consume Overlay Tiles.

They never determine lifecycle independently.

Input Components

Purpose.

Capture user interaction.

Examples include:

- text fields
- sliders
- toggles
- selectors

Input Components communicate behavioural intent.

They never directly mutate runtime behaviour.

Utility Components

Purpose.

Implement supporting presentation.

Examples include:

- spacers
- dividers
- placeholders

- skeletons
- scroll containers

Utility Components should remain behaviourally invisible.

Behavioural Neutrality

Components intentionally avoid behavioural names.

Incorrect.

```
[text]  
PlaybackButton
```

Preferred.

```
[text]  
Action Component
```

The behavioural meaning already exists within the resolved Tile.

Component Identity

Components possess stable implementation identities.

Example.

```
[text]  
Text Component
```

may render:

- Heading
- Supporting
- Caption

The Typography Resolver determines the difference.

The Component remains identical.

Component Reuse

Every Component should implement many Tile families.

Example.

A Text Component may appear inside:

- Hero Tile
- Metadata Tile
- Relationship Tile
- Utility Tile

Reuse should naturally emerge from runtime resolution.

Adaptive Behaviour

Components adapt through resolved contracts.

Desktop.



Expanded Container.

Phone.



Compact Container.

Television.



Distance-optimised Container.

The Component family remains unchanged.

Material Awareness

Components consume resolved Material Profiles.

Examples.

Hero Component.



Hero Material.

Overlay Component.



Overlay Material.

Container Component.



Surface Material.

Material behaviour belongs to the Material System.

Typography Awareness

Text Components consume Typography Profiles.

Examples.

Heading.



Resolved Typography.



Text Component.

Editorial hierarchy remains external.

Motion Awareness

Components execute Motion Profiles.

Examples.

Hero Motion.



Component Animation.

The Motion System defines behaviour.

The Component performs it.

Accessibility

Components consume Accessibility Profiles automatically.

Examples.

- larger typography
- reduced motion
- higher contrast
- accessible interaction

Accessibility should never require independent component logic.

Runtime Independence

Components should remain completely unaware of:

- Runtime World
- Expressions
- Behaviour
- Composition

They receive only resolved presentation contracts.

This keeps implementation clean and predictable.

Plugins

Extensions never provide Components.

Platform implementations provide Components.

Extensions contribute runtime behaviour only.

This preserves one coherent implementation vocabulary across the ecosystem.

Good Examples

Hero Tile

↓

Container Component

↓

Media Component

↓

Text Component

↓

Action Component

Each Component performs one implementation responsibility.

Timeline Tile

↓

Indicator Component

↓

Text Component

↓

Container Component

Behaviour remains external.

Relationship Tile

↓

Media Component

↓

Text Component



Action Component

The implementation remains reusable.

Anti-patterns

Feature Components

Creating MovieCard or AnimeCard components.

Behaviour Components

Components owning runtime behaviour.

Platform Components

Different frameworks inventing different component vocabularies.

Smart Components

Components resolving Materials, Motion or Typography independently.

Component Taxonomy Model

[mermaid]
flowchart TD

```
ResolvedTile
ResolvedTile --> Container
ResolvedTile --> Media
ResolvedTile --> Text
ResolvedTile --> Action
ResolvedTile --> Indicator
ResolvedTile --> Navigation
ResolvedTile --> Overlay
ResolvedTile --> Input
ResolvedTile --> Utility
Container --> Rendering
Media --> Rendering
Text --> Rendering
Action --> Rendering
Indicator --> Rendering
Navigation --> Rendering
Overlay --> Rendering
Input --> Rendering
Utility --> Rendering
```

Tiles determine presentation.

Components determine implementation.

Relationship To Future Chapters

The next chapter defines **Component Contracts**.

Component Taxonomy explains:

Which implementation primitives exist.

Component Contracts explain:

How every Component receives resolved runtime behaviour without introducing independent architectural decisions.

Together they establish the implementation architecture of the Component Library.

Summary

The Component Taxonomy intentionally remains:

- small,
- reusable,
- implementation focused.

Components are not behavioural concepts.

They are implementation primitives.

Every behavioural decision has already been made before Components exist.

That simplicity is the defining architectural strength of the Mosaic Component Library.

Review Status

Status

Draft

Next File

03-component-contracts.md

Component Contracts

Purpose

Components intentionally possess almost no architectural responsibility.

Their behaviour is entirely determined by the runtime.

Component Contracts define the interface between:

- Runtime Tile Resolution
- Platform Components

A Component Contract describes everything a Component needs to render correctly.

Nothing more.

Nothing less.

Definition

Within MDS, a **Component Contract** is defined as:

The complete, immutable description of a Component's runtime behaviour, presentation and interaction supplied by the Tile Framework.

Components consume contracts.

They never construct them.

Why Contracts Exist

Traditional UI frameworks frequently allow Components to determine:

- colours,
- typography,
- spacing,
- interaction,
- motion,
- hierarchy.

This tightly couples behaviour to implementation.

Mosaic intentionally follows:

[text]
Runtime

↓

Component Contract

↓

Component

↓

Rendering

Behaviour remains external.

Contracts Are Immutable

A Component Contract should never be modified by a Component.

Conceptually.

[text]
Resolved Tile

↓

Component Contract

↓

Read Only

↓

Component

Components render.

They do not negotiate.

One Responsibility

Every Component Contract communicates one implementation responsibility.

Examples.

[text]
Text Component

↓

Typography Contract

[text]
Media Component

↓

Media Contract

[text]
Action Component

↓

Interaction Contract

Responsibilities remain intentionally isolated.

Contract Inputs

Component Contracts originate from:

[text]

Resolved Tile

↓

Material Profile

↓

Typography Profile

↓

Motion Profile

↓

Interaction Profile

↓

Accessibility Profile

Components receive the completed result.

Contract Outputs

A Component Contract exposes implementation information such as:

- Material Profile
- Typography Profile
- Motion Profile
- Interaction Behaviour
- Accessibility Behaviour
- Layout Constraints
- Rendering Metadata

It intentionally excludes:

- Behaviour
- Runtime World
- Expressions
- Composition

Those concepts remain upstream.

Material Contract

Material behaviour is fully resolved before reaching Components.

Example.

[text]
Hero Tile

↓

Hero Material

↓

Material Contract

↓

Hero Component

Components should never select Materials independently.

Typography Contract

Typography follows the same model.

Example.

[text]
Heading

↓

Typography Contract

↓

Text Component

The Component simply renders the supplied typography.

Motion Contract

Motion Contracts communicate:

- sequencing
- timing
- interpolation
- lifecycle events

Components execute motion.

They never invent it.

Interaction Contract

Interaction Contracts expose:

- interaction intent

- enabled state
- focus behaviour
- accessibility actions

Platform-specific events:

- pointer
- touch
- keyboard
- remote
- voice

are mapped onto the same behavioural contract.

Accessibility Contract

Accessibility behaviour is resolved centrally.

Examples include:

- large typography
- reduced motion
- semantic labels
- accessible focus

Components should never independently implement accessibility policy.

They simply honour the Accessibility Contract.

Layout Contract

Adaptive Layout supplies:

- constraints
- available regions
- preferred sizing
- adaptive behaviour

Components should never calculate behavioural layout.

They receive already resolved constraints.

Rendering Metadata

Future implementations may expose additional rendering metadata.

Examples include:

- virtualisation hints
- image priorities
- placeholder behaviour
- lazy loading hints

These remain implementation metadata.

They should never alter behavioural presentation.

Contract Stability

Component Contracts should remain stable across platforms.

Flutter.

↓

Same Contract.

Web.

↓

Same Contract.

SwiftUI.

↓

Same Contract.

Compose.

↓

Same Contract.

Only implementation differs.

Deterministic Contracts

Given identical:

- Resolved Tile
- Runtime Profiles
- Accessibility

the Component Contract should always be identical.

Deterministic contracts simplify:

- testing
- replay
- caching

- rendering

Predictability remains the primary objective.

Stateless Components

Components should consume Contracts without introducing additional behavioural state.

Preferred.

[text]
Contract

↓

Render

Avoid.

[text]
Contract

↓

Component State

↓

Render

Runtime owns behaviour.

Components remain passive.

Runtime Updates

When behaviour changes:

[text]
New Contract

↓

Same Component

↓

Updated Presentation

Whenever practical...

Components should remain alive while Contracts evolve.

This preserves rendering efficiency and visual continuity.

Platform Adaptation

Platforms may interpret Contracts differently.

Examples.

Desktop.



Pointer interaction.

Phone.



Touch interaction.

Television.



Remote interaction.

The behavioural contract remains identical.

Only implementation differs.

Plugins

Extensions never provide Component Contracts.

Plugins contribute:

- Expressions
- Behaviour
- Information

The Runtime resolves:

- Tiles
- Contracts
- Presentation

Every extension therefore inherits identical implementation behaviour.

Good Examples

Hero

Resolved Hero Tile.



Component Contract.



Hero Component.



Rendering.

Behaviour remains external.

Timeline

Timeline Contract.

↓

Indicator Component.

↓

Presentation.

The Component performs no behavioural reasoning.

Metadata

Metadata Contract.

↓

Text Component.

↓

Rendering.

Editorial hierarchy remains runtime owned.

Anti-patterns

Smart Contracts

Contracts containing behavioural logic.

Component Decisions

Components overriding Material or Typography Contracts.

Platform Contracts

Different platforms inventing different runtime contracts.

Mutable Contracts

Components modifying runtime contracts during rendering.

Component Contract Model

[mermaid]
flowchart TD

```
ResolvedTile
ResolvedTile --> ComponentContract
ComponentContract --> MaterialProfile
ComponentContract --> TypographyProfile
ComponentContract --> MotionProfile
ComponentContract --> InteractionProfile
ComponentContract --> AccessibilityProfile
MaterialProfile --> Component
TypographyProfile --> Component
MotionProfile --> Component
InteractionProfile --> Component
AccessibilityProfile --> Component
Component
Component --> Rendering
```

The runtime defines behaviour.

Contracts communicate it.

Components implement it.

Relationship To Future Chapters

The next chapter defines **Component Lifecycle**.

Component Contracts explain:

What Components receive.

Component Lifecycle explains:

How Components evolve while consuming changing Contracts throughout the lifetime of the Runtime World.

Together they establish the implementation model of the Component Library.

Summary

Component Contracts complete one of the most important architectural separations within Mosaic.

Runtime determines behaviour.

Contracts communicate behaviour.

Components implement behaviour.

By preserving this separation, Mosaic ensures that rendering technologies remain replaceable while behavioural understanding remains permanently stable.

Review Status

Status

Draft

Next File

04-component-lifecycle.md

Component Lifecycle

Purpose

The Tile Framework defines the behavioural lifecycle.

The Component Library defines the implementation lifecycle.

Unlike Tiles, which preserve behavioural identity over time, Components are intentionally disposable implementation objects.

This distinction is one of the most important architectural separations within Mosaic.

Users experience persistent Tiles.

The runtime manages transient Components.

Definition

Within MDS, **Component Lifecycle** is defined as:

The implementation lifecycle through which Components render evolving Component Contracts while remaining behaviourally passive.

Component lifecycle concerns implementation.

Behaviour remains entirely external.

Philosophy

Traditional UI frameworks frequently merge:

- behavioural lifetime,
- component lifetime.

Mosaic intentionally separates them.

[text]
Tile

↓

Persistent Behaviour

↓

Component

↓

Disposable Implementation

Components may be:

- recreated,
- recycled,

- virtualised,
- pooled.

Behavioural continuity remains unaffected because the Tile survives.

Lifecycle Stages

Every Component progresses through the same conceptual lifecycle.

[text]
Created

↓

Bound

↓

Rendered

↓

Updated

↓

Detached

↓

Released

Each stage communicates one implementation responsibility.

Stage One

Created

The Component instance is allocated.

At this stage it possesses:

- implementation identity,
- no behavioural information,
- no presentation.

Creation should remain inexpensive.

Stage Two

Bound

The Component receives a Component Contract.

Examples include:

- Material Profile
- Typography Profile
- Motion Profile
- Accessibility Profile

Only after binding does the Component understand how it should render.

Stage Three

Rendered

The Component becomes visible.

Rendering should simply express the current Contract.

No behavioural decisions should occur during this stage.

Stage Four

Updated

Most Components spend the majority of their lifetime receiving updated Contracts.

Preferred.

[text]
New Contract

↓

Existing Component

↓

Updated Rendering

Avoid.

[text]
Destroy Component

↓

Create Component

Updating preserves rendering efficiency while maintaining behavioural continuity.

Stage Five

Detached

The Component is removed from the active presentation.

Detachment does **not** necessarily imply destruction.

Future implementations may:

- pool,
- recycle,
- virtualise,

detached Components.

Stage Six

Released

The implementation object is discarded.

Importantly...

The Tile may still exist.

Behavioural lifetime and Component lifetime intentionally remain independent.

Contracts Drive Lifecycle

Components should evolve only because their Contracts evolve.

Incorrect.

[text]
Component

↓

Internal State

↓

Rendering

Correct.

[text]
Updated Contract

↓

Updated Component

The runtime owns behavioural evolution.

Stateless Rendering

Whenever practical...

Components should remain stateless.

Preferred.

[text]
Contract

↓

Render

Avoid.

[text]
Contract

↓

Mutable Component State

↓

Render

State belongs upstream.

Reuse

Future implementations should aggressively reuse Components.

Examples include:

- scrolling collections,
- long libraries,
- virtual lists,
- infinite grids.

Reuse is an implementation optimisation.

Behaviour remains entirely unchanged.

Identity

Component identity should never be exposed to users.

Users should perceive:

Hero Tile

Not:

Hero Component Instance

Component identity is an implementation detail.

Tile identity remains behaviourally meaningful.

Material Updates

Material changes should normally produce:

[text]
Updated Material Contract

↓

Same Component



Updated Material Rendering

Component replacement should rarely be necessary.

Typography Updates

Typography behaves similarly.

Accessibility changes.



Typography Contract.



Text Component updates.



Reading continues.

The runtime should preserve continuity wherever practical.

Motion Updates

Motion should also follow Contracts.

Components execute:

- Motion start,
- Motion update,
- Motion completion.

They never determine Motion sequencing independently.

Accessibility Updates

Accessibility should update Components through new Contracts.

Examples.

Reduced Motion.



Motion Contract updates.

Large Text.



Typography Contract updates.

High Contrast.



Material Contract updates.

The Component simply re-renders.

Adaptive Updates

Adaptive behaviour also updates existing Components.

Examples.

Window resized.



Adaptive Contract.



Existing Component updates.

The lifecycle should remain continuous.

Runtime Ownership

The Runtime owns:

- Contracts,
- lifecycle timing,
- updates.

Components own:

- rendering,
- resource usage,
- implementation.

Responsibilities remain intentionally separated.

Failure Behaviour

If a Component fails.

Preferred.

[text]
Fallback Component



Continue Rendering

Avoid.

[text]
Rendering Stops

↓

Behaviour Lost

Implementation failures should not compromise runtime understanding wherever practical.

Platform Independence

Every platform should follow the same lifecycle.

Flutter.

↓

Created.

↓

Bound.

↓

Rendered.

↓

Updated.

↓

Released.

React.

↓

Same lifecycle.

SwiftUI.

↓

Same lifecycle.

Behaviour remains platform independent.

Plugins

Extensions never participate in Component lifecycle.

Plugins contribute:

- behaviour,
- Expressions,
- information.

The platform owns:

- Components,
- Contracts,
- rendering.

Every extension therefore automatically inherits future implementation improvements.

Good Examples

Playback

Hero Component.

↓

New Contract.

↓

Updated Hero.

↓

Playback continues.

No behavioural interruption occurs.

Reading

Typography updates.

↓

Text Component updates.

↓

Reader continues naturally.

Library

Scrolling.

↓

Components recycled.

↓

Tiles remain behaviourally identical.

Users perceive one continuous library.

Anti-patterns

Behaviour Components

Components storing runtime behaviour.

Component Ownership

Components mutating the Runtime World.

Replacement Rendering

Destroying Components for every runtime update.

Platform Lifecycles

Different clients inventing incompatible lifecycle models.

Component Lifecycle Model

[mermaid]
flowchart TD

```
Created
Created --> Bound
Bound --> Rendered
Rendered --> Updated
Updated --> Detached
Detached --> Released
```

The Component lifecycle exists purely to implement changing Contracts.

Behaviour remains external.

Relationship To Future Chapters

The next chapter defines **Component Composition**.

Component Lifecycle explains:

How Components evolve over time.

Component Composition explains:

How Components combine to faithfully implement resolved Tiles while preserving the behavioural architecture established throughout Mosaic.

Together they complete the implementation lifecycle of the Component Library.

Summary

Component Lifecycle intentionally separates implementation from behaviour.

Tiles persist.

Components do not.

Contracts evolve.

Components render.

This separation allows Mosaic to optimise rendering aggressively without ever compromising behavioural continuity or architectural clarity.

Review Status

Status

Draft

Next File

05-component-composition.md

Component Composition

Purpose

Individual Components rarely exist alone.

A Hero Tile may require:

- Container Component,
- Media Component,
- Text Component,
- Action Component.

Component Composition defines how these implementation primitives combine to faithfully render one resolved Tile.

Unlike the Tile Framework, Component Composition is **not** concerned with behavioural grouping.

That work has already been completed.

Its responsibility is purely implementation.

Definition

Within MDS, **Component Composition** is defined as:

The deterministic assembly of implementation Components required to render one resolved Tile while preserving the behavioural intent established by the runtime.

Component Composition concerns implementation.

Behaviour remains external.

Philosophy

Traditional UI frameworks frequently begin by composing widgets.

Examples.

[text]
Card

↓

Image

↓

Text

↓

Buttons

Mosaic intentionally begins one layer higher.

[text]
Resolved Tile

↓

Component Composition

↓

Platform Rendering

The Tile determines composition.

Components implement it.

Behaviour Before Composition

Components should never determine what belongs together.

Incorrect.

[text]
Media

↓

Text

↓

Looks Correct

Correct.

[text]
Hero Tile

↓

Resolved Contract

↓

Component Composition

Runtime understanding always precedes implementation.

One Tile

Every Component Composition should implement one Tile.

Example.

[text]
Hero Tile

↓

Container

↓

Media

↓

Text

↓

Action

The Tile remains the behavioural unit.

Components remain implementation details.

Composition Inputs

Component Composition consumes:

[text]

Resolved Tile

↓

Material Contract

↓

Typography Contract

↓

Motion Contract

↓

Interaction Contract

↓

Accessibility Contract

Every required decision has already been made.

Composition Outputs

Component Composition produces:

[text]

Platform Component Tree

↓

Rendering Tree

↓

Accessibility Tree

Behaviour does not appear within these outputs.

Container First

Most Component Compositions begin with a Container Component.

Purpose.

Provide:

- structure,
- clipping,
- spacing,
- Material implementation.

The Container should never introduce behavioural meaning.

Media Composition

Media Components render:

- artwork,
- thumbnails,
- video,
- imagery.

Media Components should never determine:

- hierarchy,
- Hero status,
- interaction.

Those responsibilities belong to the Tile.

Typography Composition

Text Components render resolved Typography Contracts.

Example.

[text]
Heading

↓

Text Component

Typography should never be chosen by Component Composition.

The runtime already resolved it.

Action Composition

Action Components render behavioural intent.

Example.

[text]
Action Tile

↓

Interaction Contract

↓

Action Component

Buttons remain implementation details.

Interaction remains behavioural.

Indicator Composition

Indicators render resolved state.

Examples.

- playback progress,
- download progress,
- activity.

Indicators should never compute state independently.

They simply display resolved values.

Nested Composition

Components may contain other Components.

Example.

[text]
Container

■■■ Media

■■■ Text

■■■ Action

Nested composition improves implementation.

It should never alter Tile identity.

Material Cohesion

Every Component within one Tile should consume compatible Material Contracts.

Example.

Hero Tile.

↓

Hero Material.

↓

Every child Component inherits compatible Material behaviour.

Components should never introduce conflicting Materials.

Typography Cohesion

Editorial hierarchy should remain intact.

Example.

[text]
Heading

↓

Supporting

↓

Caption

Component Composition should preserve this ordering exactly.

Motion Cohesion

Components should execute one Motion Profile.

Example.

Hero Tile.

↓

Hero Motion.

↓

Every child Component participates.

Movement should appear unified.

Not individually animated.

Accessibility Cohesion

Accessibility Contracts should apply consistently across every Component within a Tile.

Example.

Large text.

↓

Every Text Component updates.

Reduced motion.

↓

Every child Component respects Motion Contracts.

Accessibility should remain behaviourally coherent.

Adaptive Composition

Adaptive behaviour may alter Component Composition.

Example.

Desktop.

↓

Media.

↓

Text.

↓

Actions.

Phone.

↓

Media.

↓

Text.

↓

Collapsed Actions.

The behavioural Tile remains identical.

Only implementation changes.

Runtime Updates

Updated Contracts should generally update existing Components.

Preferred.

[text]
New Contract

↓

Existing Component Tree

↓

Updated Rendering

Avoid.

[text]
Destroy Tree

↓

Create Tree

Stable implementation improves performance while preserving continuity.

Platform Independence

Every platform should construct equivalent Component trees.

Flutter.

↓

Hero Components.

React.

↓

Equivalent Hero Components.

SwiftUI.

↓

Equivalent Hero Components.

Implementation differs.

Behaviour remains identical.

Plugins

Extensions never define Component Composition.

Plugins contribute:

- behaviour,
- Expressions.

The Tile Framework resolves presentation.

Platform implementations compose Components.

Every extension therefore automatically inherits native implementation quality.

Good Examples

Hero

Resolved Hero Tile.

↓

Container.

↓

Media.

↓

Text.

↓

Action.

↓

Rendering.

Everything reflects one behavioural contract.

Timeline

Timeline Tile.

↓

Container.

↓

Indicator.

↓

Text.

Behaviour remains runtime owned.

Metadata

Metadata Tile.

↓

Container.

↓

Text.

↓

Indicator.

Editorial hierarchy remains intact.

Anti-patterns

Smart Composition

Component trees determining behaviour.

Platform Composition

Different clients inventing different Component structures.

Behavioural Containers

Containers mutating runtime state.

Component Hierarchy

Implementation structure redefining behavioural hierarchy.

Component Composition Model

[mermaid]
flowchart TD

```
ResolvedTile
ResolvedTile --> ComponentContracts
ComponentContracts --> Container
Container --> Media
Container --> Text
Container --> Action
Container --> Indicator
Media --> Rendering
Text --> Rendering
Action --> Rendering
Indicator --> Rendering
```

Resolved Tiles determine composition.

Components faithfully implement it.

Relationship To Future Chapters

The next chapter defines **Rendering Architecture**.

Component Composition explains:

How Components combine into implementation.

Rendering Architecture explains:

How those Components become efficient, deterministic presentation across every supported rendering technology.

Together they complete the implementation architecture of the Component Library.

Summary

Component Composition intentionally remains simple.

It assembles Components.

It does not solve behaviour.

Every Component tree should faithfully reflect:

- one Tile,
- one behavioural intent,
- one runtime contract.

By keeping implementation downstream from behaviour, Mosaic ensures that rendering technology can evolve indefinitely without changing the architectural language of the platform.

Review Status

Status

Draft

Next File

06-rendering-architecture.md

Rendering Architecture

Purpose

The Component Library concludes with one responsibility.

Rendering.

Everything before this chapter has progressively transformed:

- behaviour,
- understanding,
- presentation,

into resolved Component Contracts.

Rendering Architecture defines how those resolved Components become pixels without introducing new architectural decisions.

Rendering should be considered the thinnest layer in the entire Mosaic architecture.

Its responsibility is implementation.

Nothing more.

Definition

Within MDS, **Rendering Architecture** is defined as:

The deterministic implementation layer responsible for transforming resolved Component Contracts into visible presentation while remaining behaviourally passive.

Rendering implements.

It never reasons.

Philosophy

Traditional UI architectures frequently combine:

- rendering,
- state,
- behaviour,
- layout.

Mosaic intentionally separates these responsibilities.

[text]

Runtime World

↓

Composition

↓

Tiles

↓

Components

↓

Rendering

Rendering becomes the final implementation stage.

Nothing upstream depends upon it.

Rendering Is Passive

Rendering should never ask:

What should I display?

Instead it should ask:

How should I faithfully display what has already been resolved?

Every rendering decision should originate from Component Contracts.

Rendering Pipeline

Every rendered frame follows the same conceptual pipeline.

[text]

Resolved Tile

↓

Component Contract

↓

Platform Component

↓

Rendering

↓

Presentation

Every upstream decision has already been made.

Rendering simply implements it.

Rendering Inputs

Rendering consumes:

[text]

Platform Components

↓

Material Profiles

↓

Typography Profiles

↓

Motion Profiles

↓

Interaction Profiles

↓

Accessibility Profiles

Rendering never consumes:

- Runtime World
- Behaviour
- Expressions
- Runtime Hierarchy

Those concepts remain architecturally upstream.

Rendering Outputs

Rendering produces:

- pixels,
- accessibility trees,
- input regions,
- compositor layers.

These outputs remain implementation artefacts.

They should never influence runtime behaviour.

Rendering Order

Rendering should respect runtime hierarchy.

Conceptually.

[text]
Canvas

↓

Supporting

↓

Hero

↓

Over lay

This ordering originates from the Material System.

Rendering simply preserves it.

Material Rendering

Material behaviour should already be resolved.

Rendering implements:

- Acrylic
- Hero Materials
- Overlay Materials
- Refraction
- Runtime Atmosphere

Rendering should never reinterpret Material behaviour independently.

Typography Rendering

Typography rendering should consume resolved typography.

Examples.

[text]
Heading

↓

Text Rendering

[text]
Supporting

↓

Text Rendering

The rendering layer should never choose:

- weights,
- hierarchy,
- editorial emphasis.

Those decisions already exist.

Motion Rendering

Motion Profiles should be executed faithfully.

Rendering should perform:

- interpolation,
- frame scheduling,
- compositing,
- animation playback.

Rendering should never modify behavioural sequencing.

Interaction Rendering

Rendering exposes interaction surfaces.

Examples.

- touch regions
- pointer regions
- focus rings
- accessibility actions

Behaviour remains unchanged.

Rendering simply exposes implementation.

Incremental Rendering

Rendering should update only affected regions whenever practical.

Preferred.

[text]
Timeline

↓

Render Timeline

Avoid.

[text]
Timeline

↓

Render Entire Screen

Incremental rendering preserves performance without affecting runtime behaviour.

Layering

Future implementations may internally use rendering layers.

Examples.

[text]
Canvas Layer

↓

Material Layer

↓

Content Layer

↓

Overlay Layer

Layering remains an implementation concern.

Behaviour should remain completely unaware of compositor architecture.

GPU Independence

The Rendering Architecture intentionally avoids depending upon:

- GPU APIs,
- shader languages,
- graphics libraries.

Future implementations may use:

- Skia
- Metal
- Vulkan
- DirectX
- WebGPU
- OpenGL
- Canvas

Rendering technology remains replaceable.

Architectural behaviour does not.

Performance

Rendering should optimise:

- batching,
- caching,
- layer reuse,
- partial invalidation,
- GPU utilisation.

Optimisation should never modify:

- hierarchy,
- Materials,
- Motion,
- Typography,
- interaction.

Correctness remains the highest priority.

Accessibility

Accessibility rendering should consume resolved Accessibility Profiles.

Examples.

- semantic trees
- screen reader metadata
- focus order
- contrast adjustments

Rendering should never invent accessibility behaviour independently.

Deterministic Rendering

Given identical:

- Component Contracts,
- runtime profiles,
- rendering capabilities,

Rendering should produce identical presentation.

Deterministic rendering improves:

- testing,
- screenshots,
- replay,
- debugging.

Platform Independence

Different rendering engines may produce different implementations.

Flutter.

↓

Impeller.

Web.



Canvas/WebGPU.

SwiftUI.



Core Animation.

Compose.



Skia.

Presentation should remain behaviourally identical.

Only rendering implementation changes.

Failure Behaviour

Rendering failures should degrade gracefully.

Preferred.

[text]
Material Effect Fails



Fallback Material



Continue Rendering

Avoid.

[text]
Rendering Failure



Blank Interface

Behavioural continuity should remain the highest priority.

Plugins

Extensions never participate directly in rendering.

Plugins contribute:

- behaviour,
- information,
- Expressions.

Rendering remains entirely platform owned.

Every extension therefore automatically inherits future rendering improvements.

Good Examples

Hero

Resolved Hero Tile.

↓

Hero Component.

↓

Rendering.

↓

Presentation.

Every architectural decision already exists before rendering begins.

Playback

Timeline updates.

↓

Timeline Component.

↓

Partial rendering.

↓

Presentation.

Only affected regions redraw.

Reading

Typography changes.

↓

Text Component.

↓

Incremental rendering.

↓

Reader continues naturally.

Anti-patterns

Smart Rendering

Rendering deciding behaviour.

Rendering State

Graphics systems mutating runtime architecture.

Platform Behaviour

Different renderers creating different runtime experiences.

Behavioural Shaders

GPU effects changing behavioural understanding.

Rendering Architecture Model

[mermaid]
flowchart TD

```
ResolvedTile
ResolvedTile --> ComponentContract
ComponentContract --> PlatformComponent
PlatformComponent --> Rendering
Rendering --> Presentation
Presentation --> User
```

Rendering faithfully implements architectural intent.

Nothing upstream depends upon rendering.

Relationship To Future Chapters

The next chapter defines **Platform Components**.

Rendering Architecture explains:

How Components become visible.

Platform Components explain:

How different UI frameworks implement identical Component Contracts while preserving one behavioural language.

Together they establish the implementation foundation of Mosaic.

Summary

Rendering is intentionally the simplest architectural layer in Mosaic.

By the time rendering begins:

- behaviour has been solved,
- presentation has been solved,
- Components have been resolved.

Rendering simply makes those decisions visible.

That disciplined separation is what allows Mosaic to evolve for decades while remaining behaviourally consistent across every future platform.

Review Status

Status

Draft

Next File

07-platform-components.md

Platform Components

Purpose

The Component Library defines one implementation vocabulary.

Platform Components express that vocabulary using the capabilities of each supported UI framework.

Unlike traditional cross-platform architectures, Mosaic does **not** attempt to make every framework identical.

Instead it requires every framework to faithfully implement the same behavioural contracts.

Different implementation.

One behavioural language.

Definition

Within MDS, **Platform Components** are defined as:

Platform-specific implementations of Mosaic Component Contracts that preserve identical behavioural, visual and accessibility semantics across every supported client.

Platform Components implement.

They never reinterpret.

Philosophy

Every rendering platform provides different capabilities.

Examples include:

- Flutter
- Web
- SwiftUI
- Jetpack Compose
- Future rendering engines

These frameworks are implementation details.

The user should always experience:

- the same hierarchy,
- the same Materials,
- the same Typography,
- the same Motion,
- the same interaction.

Changing framework should never change behaviour.

One Contract

Every Platform Component consumes the same Component Contract.

Conceptually.

[text]
Resolved Tile

↓

Component Contract

↓

Flutter Component

[text]
Resolved Tile

↓

Component Contract

↓

React Component

[text]
Resolved Tile

↓

Component Contract

↓

SwiftUI Component

The implementation differs.

The contract does not.

Platform Responsibilities

Each platform implementation is responsible for:

- rendering
- input handling
- accessibility APIs
- graphics integration
- lifecycle integration
- platform optimisation

Platforms are **not** responsible for:

- runtime behaviour

- hierarchy
- Materials
- Typography
- Motion sequencing

Those responsibilities remain upstream.

Flutter

Flutter Components should:

- consume Component Contracts directly,
- use Impeller where available,
- implement Material behaviour faithfully,
- preserve runtime Motion ordering.

Flutter should never redefine behavioural presentation because framework APIs differ.

Web

Web Components should:

- consume Component Contracts,
- implement Material behaviour using appropriate web technologies,
- preserve accessibility semantics,
- maintain deterministic rendering.

CSS, Canvas or WebGPU remain implementation choices.

Not architectural concepts.

SwiftUI

SwiftUI Components should:

- consume Component Contracts,
- preserve behavioural hierarchy,
- integrate naturally with Apple accessibility,
- respect resolved Motion Profiles.

Native platform conventions should never weaken Mosaic behaviour.

Jetpack Compose

Compose Components should:

- faithfully implement runtime Contracts,
- preserve Material behaviour,
- maintain Motion sequencing,
- expose identical accessibility semantics.

Platform differences should remain invisible to users.

Future Platforms

Future platforms should require only:

[text]
Component Contract

↓

Platform Adapter

↓

Rendering

The architecture intentionally assumes that rendering technologies will continue evolving.

The Component Library should remain stable despite those changes.

Material Fidelity

Different platforms may render Materials differently.

Desktop GPU.

↓

Highest fidelity.

Phone.

↓

Balanced fidelity.

Low-power device.

↓

Simplified Acrylic.

Material identity should remain recognisable.

Only rendering quality changes.

Typography Fidelity

Typography should preserve:

- editorial hierarchy,
- reading rhythm,
- accessibility,
- optical consistency.

Font technologies may differ.

Editorial language must not.

Motion Fidelity

Platforms may implement Motion differently.

Examples.

Flutter.



Physics.

Web.



CSS interpolation.

SwiftUI.



Native animation engine.

Motion ordering should remain identical.

Perceptual behaviour matters more than implementation technique.

Interaction Fidelity

Platform Components should expose identical behavioural interaction.

Examples.

Touch.



Behaviour.

Pointer.



Behaviour.

Remote.



Behaviour.

Voice.



Behaviour.

Different input systems.

One runtime outcome.

Accessibility Fidelity

Platform Components should integrate naturally with platform accessibility APIs.

Examples include:

- VoiceOver
- TalkBack
- Narrator
- browser accessibility trees

The user should receive equivalent behavioural understanding regardless of platform.

Runtime Synchronisation

Platform Components should remain synchronised with Runtime Contracts.

Contract updates.



Platform updates.



Presentation updates.

Platform-specific state should never diverge from runtime state.

Performance

Platform Components should optimise:

- recomposition,
- virtualisation,
- rendering,

- memory usage,
- GPU utilisation.

Performance optimisation should never alter behavioural correctness.

Deterministic Behaviour

Given identical:

- Component Contracts,
- runtime state,
- accessibility,

Platform Components should produce equivalent presentation across every supported client.

Perfect pixel parity is unnecessary.

Behavioural parity is mandatory.

Plugins

Extensions never implement Platform Components.

Plugins contribute:

- behaviour,
- Expressions.

The Component Library provides platform implementations.

Every extension therefore automatically supports every Mosaic client.

Good Examples

Flutter

Component Contract.

↓

Flutter Hero Component.

↓

Rendering.

↓

Behaviour preserved.

Web

Component Contract.



Web Timeline Component.



Rendering.



Behaviour preserved.

Television

Component Contract.



TV Overlay Component.



Rendering.



Distance-optimised presentation.

The behavioural language remains unchanged.

Anti-patterns

Platform Behaviour

Flutter behaving differently from Web.

Framework Contracts

Platform APIs redefining Component Contracts.

Native Drift

Platform conventions replacing Mosaic interaction.

Extension Components

Plugins shipping platform-specific UI implementations.

Platform Component Model

[mermaid]
flowchart TD

```
ComponentContract
ComponentContract --> PlatformAdapter
PlatformAdapter --> Flutter
PlatformAdapter --> Web
PlatformAdapter --> SwiftUI
PlatformAdapter --> Compose
PlatformAdapter --> FuturePlatforms
Flutter --> Presentation
Web --> Presentation
SwiftUI --> Presentation
Compose --> Presentation
FuturePlatforms --> Presentation
```

One Component Contract.

Many implementations.

One behavioural experience.

Relationship To Future Chapters

The next chapter defines **Accessibility Contracts**.

Platform Components explain:

How Component Contracts become platform implementations.

Accessibility Contracts explain:

How accessibility becomes a first-class implementation responsibility while remaining behaviourally identical across every platform.

Together they ensure Mosaic remains both consistent and inclusive.

Summary

Platform Components intentionally separate:

- behavioural architecture,
- implementation technology.

Frameworks evolve.

Platforms change.

Rendering engines come and go.

The Component Contract remains stable.

Users should therefore experience the same Companion regardless of which device or framework happens to be rendering it.

Review Status

Status

Draft

Next File

08-accessibility-contracts.md

Accessibility Contracts

Purpose

Accessibility should never be treated as an optional implementation layer.

By the time a Component renders, accessibility has already been resolved by the runtime.

Accessibility Contracts communicate that resolved behaviour to platform Components.

They ensure every implementation preserves:

- behavioural understanding,
- hierarchy,
- interaction,
- presentation,

without requiring Components to invent accessibility behaviour independently.

Definition

Within MDS, **Accessibility Contracts** are defined as:

Immutable runtime contracts describing the accessibility behaviour every Component must implement while preserving behavioural parity with the standard presentation model.

Accessibility Contracts describe implementation.

They never redefine behaviour.

Philosophy

Traditional UI frameworks often require each Component to implement accessibility independently.

This frequently results in:

- inconsistent screen reader behaviour,
- incomplete keyboard navigation,
- different accessibility across platforms.

Mosaic intentionally resolves accessibility centrally.

[text]
Runtime

↓

Accessibility Contract

↓

Platform Component

↓

Accessible Presentation

Behaviour remains identical.

Only implementation changes.

Accessibility Before Components

Accessibility should always be resolved before Components exist.

Incorrect.

[text]
Component

↓

Accessibility Logic

Correct.

[text]
Runtime

↓

Accessibility Contract

↓

Component

Components consume accessibility.

They do not create it.

Contract Inputs

Accessibility Contracts are generated from:

[text]
Resolved Tile

↓

Accessibility Profile

↓

Interaction Profile

↓

Typography Profile

↓

Material Profile

All behavioural decisions have already been made.

Accessibility refines presentation.

Contract Outputs

Accessibility Contracts communicate:

- semantic role
- accessible name
- accessible description
- interaction actions
- focus behaviour
- reading order
- motion behaviour
- contrast behaviour

Every platform consumes the same conceptual contract.

Semantic Roles

Every Component should expose one semantic role.

Examples include:

[text]
Heading

Button

Progress

Navigation

Image

List

Dialog

Article

Semantic roles originate from resolved runtime behaviour.

Not from component implementation.

Accessible Names

Every interactive Component should expose an accessible name.

Example.

[text]
Resume Tile

↓

Resume playback

Not.

[text]
Button

Names communicate behavioural purpose.

Not implementation.

Accessible Descriptions

Additional context should remain available when required.

Examples.

Continue watching.

Episode 18.

32 minutes remaining.

Descriptions should enrich understanding.

Never replace it.

Focus Behaviour

Focus order should follow Runtime Hierarchy.

Preferred.

[text]
Hero

↓

Primary

↓

Supporting

↓

Peripheral

Focus should never depend upon rendering order alone.

Behavioural hierarchy remains authoritative.

Reading Order

Screen readers should communicate:

- editorial hierarchy,
- behavioural grouping,
- relationships,

rather than raw component structure.

Reading order should follow the same understanding experienced visually.

Interaction Actions

Accessibility Contracts expose behavioural actions.

Examples.

[text]
Activate

Resume

Bookmark

Download

Open

Dismiss

Platform-specific gestures invoke these actions.

Behaviour remains platform independent.

Typography Accessibility

Typography adjustments may include:

- larger text
- optical sizing
- stronger grade
- additional spacing

Components should consume these changes automatically through Typography Contracts.

Accessibility remains runtime owned.

Material Accessibility

Material adjustments may include:

- increased contrast
- reduced translucency
- simplified refraction
- clearer separation

Material identity remains recognisable.

Only presentation changes.

Motion Accessibility

Accessibility Contracts communicate Motion preferences.

Examples.

Full Motion.

↓

Reduced Motion.

↓

Minimal Motion.

↓

Instant.

Components execute these profiles.

They never interpret accessibility preferences independently.

High Contrast

High Contrast behaviour should remain behaviourally identical.

Only presentation changes.

Examples include:

- stronger contrast
- clearer focus indicators
- simplified Materials

Hierarchy should remain unchanged.

Input Independence

Accessibility Contracts should support:

- touch
- keyboard
- pointer
- remote
- switch access
- voice

Interaction methods differ.

Behavioural meaning remains identical.

Platform Integration

Each platform should map Accessibility Contracts onto native accessibility APIs.

Examples include:

- VoiceOver
- TalkBack
- Narrator
- browser accessibility trees

Platform differences remain implementation details.

Runtime Updates

Accessibility Contracts may change during runtime.

Examples.

Large Text enabled.

↓

Typography Contract updates.

↓

Accessibility Contract updates.

↓

Component updates.

Behaviour remains unchanged.

Only presentation evolves.

Deterministic Behaviour

Given identical:

- Runtime World
- Accessibility Profile
- Resolved Tile

Accessibility Contracts should always remain identical.

Determinism improves:

- testing
- replay
- platform parity

Accessibility should be predictable.

Plugins

Extensions never implement accessibility.

Plugins contribute:

- behaviour
- information
- Expressions

The runtime produces Accessibility Contracts.

Every extension therefore automatically inherits future accessibility improvements.

Good Examples

Playback

Resume Tile.

↓

Accessibility Contract.

↓

VoiceOver.

↓

"Resume playback. Episode 18. Thirty-two minutes remaining."

Behaviour remains clear.

Reading

Chapter Tile.

↓

Large Text.

↓

Updated Typography.

↓

Reader continues naturally.

Television

Navigation Tile.

↓

Remote focus.

↓

Accessible labels.

↓

Equivalent behavioural understanding.

Anti-patterns

Component Accessibility

Components independently implementing accessibility policy.

Platform Accessibility

Different clients exposing different behavioural semantics.

Decorative Accessibility

Accessibility changing behavioural hierarchy.

Plugin Accessibility

Extensions bypassing runtime accessibility.

Accessibility Contract Model

[mermaid]
flowchart TD

```
ResolvedTile
ResolvedTile --> AccessibilityProfile
AccessibilityProfile --> AccessibilityContract
AccessibilityContract --> Semantics
AccessibilityContract --> Focus
AccessibilityContract --> Interaction
AccessibilityContract --> Motion
AccessibilityContract --> Typography
Semantics --> PlatformComponent
Focus --> PlatformComponent
Interaction --> PlatformComponent
Motion --> PlatformComponent
Typography --> PlatformComponent
PlatformComponent
PlatformComponent --> AccessiblePresentation
```

Accessibility is resolved once.

Every platform implements the same behavioural contract.

Relationship To Future Chapters

The next chapter defines **Runtime Rendering**.

Accessibility Contracts explain:

How Components become accessible.

Runtime Rendering explains:

How resolved Components are scheduled, updated and rendered efficiently while preserving behavioural and accessibility guarantees.

Together they complete the implementation pipeline of the Component Library.

Summary

Accessibility Contracts ensure every Mosaic experience remains:

- understandable,
- navigable,
- consistent,
- behaviourally identical,

regardless of:

- platform,
- input method,
- accessibility preference.

Accessibility is therefore not an additional feature.

It is part of the architectural contract every Component faithfully implements.

Review Status

Status

Draft

Next File

09-runtime-rendering.md

Runtime Rendering

Purpose

Every previous chapter has progressively transformed behavioural understanding into implementation.

The final responsibility of the Component Library is Runtime Rendering.

Runtime Rendering is responsible for:

- scheduling,
- updating,
- composing,
- presenting,

resolved Components efficiently while preserving the behavioural guarantees established throughout the Mosaic architecture.

Rendering should never become another behavioural system.

Its responsibility is execution.

Definition

Within MDS, **Runtime Rendering** is defined as:

The deterministic execution of resolved Component Contracts into continuously updated visual presentation while preserving behavioural, material and accessibility correctness.

Runtime Rendering implements.

It never decides.

Philosophy

Traditional applications frequently merge:

- rendering,
- application state,
- behaviour,
- scheduling.

Mosaic intentionally separates these concerns.

[text]
Behaviour

↓

Runtime Resolution

↓

Component Contracts

↓

Runtime Rendering

↓

Presentation

By the time rendering begins...

Every important decision has already been made.

Rendering Is Continuous

Rendering should not be viewed as a sequence of pages.

Instead it continuously reflects the evolving Runtime World.

Conceptually.

[text]
Behaviour

↓

Contracts Update

↓

Rendering Evolves

↓

Presentation Evolves

The renderer should appear alive without becoming behaviourally intelligent.

Rendering Pipeline

Every rendering update should broadly follow this pipeline.

[text]
Resolved Components

↓

Change Detection

↓

Render Scheduling

↓

Composition

↓

GPU Submission

↓

Presentation

Each stage performs one responsibility.

Change Detection

Rendering should begin only after behavioural resolution completes.

The renderer evaluates:

- updated Component Contracts,
- invalidated presentation,
- changed visual regions.

Rendering should never poll runtime systems independently.

Render Scheduling

Rendering work should be prioritised according to Runtime Hierarchy.

Preferred order.

[text]
Hero

↓

Primary

↓

Supporting

↓

Peripheral

Scheduling should reinforce behavioural importance.

It should never redefine it.

Incremental Rendering

Rendering should update only affected presentation.

Preferred.

[text]
Timeline Updated

↓

Timeline Rendered

Avoid.

[text]
Timeline Updated

↓

Entire Scene Re-rendered

Incremental rendering improves responsiveness without affecting behavioural correctness.

Composition

Runtime Rendering assembles platform Components into the final render tree.

Composition should preserve:

- Material hierarchy,
- Typography hierarchy,
- Motion sequencing,
- accessibility order.

Rendering should faithfully reproduce the Presentation Model.

Layer Composition

Future implementations may internally compose layers.

Conceptually.

[text]
Canvas

↓

Materials

↓

Content

↓

Overlay

↓

System UI

Layer composition remains an implementation concern.

The behavioural hierarchy should remain identical regardless of compositor architecture.

Material Rendering

Runtime Rendering implements resolved Materials.

Examples include:

- Acrylic
- Hero Material
- Overlay Material

- Refraction
- Runtime Atmosphere

Rendering should never reinterpret Material behaviour.

It faithfully executes resolved Material Profiles.

Typography Rendering

Typography should remain stable.

Rendering implements:

- glyph positioning,
- shaping,
- wrapping,
- optical sizing.

Editorial hierarchy should already be resolved.

Rendering simply displays it.

Motion Rendering

Motion execution should remain deterministic.

Examples include:

- interpolation,
- timeline progression,
- frame updates,
- completion.

Behaviour determines motion.

Rendering performs it.

Interaction Rendering

Runtime Rendering exposes interaction surfaces.

Examples.

- touch targets,
- pointer regions,
- accessibility regions,
- focus order.

These surfaces implement resolved Interaction Contracts.

Behavioural meaning remains unchanged.

Accessibility Rendering

Accessibility should be rendered alongside visual presentation.

Examples include:

- semantic trees,
- accessible names,
- reading order,
- focus order,
- interaction actions.

Accessible presentation should remain behaviourally identical to visual presentation.

Virtualisation

Future implementations may virtualise Components.

Examples include:

- long collections,
- library shelves,
- search results,
- recommendations.

Virtualisation is purely an implementation optimisation.

Users should never perceive behavioural differences.

Partial Invalidation

Only changed presentation should invalidate rendering.

Examples.

Playback progress.

↓

Timeline redraw.

Theme change.

↓

Material redraw.

Focus change.

↓

Hero redraw.

Behaviour determines invalidation.

Frame Consistency

Every rendered frame should represent one consistent Runtime World.

Rendering should never display:

- partially updated hierarchy,
- mixed behavioural states,
- incomplete Material updates.

Each frame should correspond to one completed runtime resolution.

Performance Profiles

Future runtime implementations may expose conceptual rendering profiles.

Examples.

[text]

High Fidelity

↓

Full Materials

↓

Full Motion

[text]

Balanced

↓

Reduced Effects

↓

Equivalent Behaviour

[text]

Efficiency

↓

Simplified Rendering

↓

Equivalent Behaviour

Rendering quality may change.

Behaviour must not.

Platform Independence

Runtime Rendering should remain implementation independent.

Flutter.

↓

Impeller.

Web.

↓

Canvas / WebGPU.

SwiftUI.

↓

Core Animation.

Compose.

↓

Skia.

Presentation should remain behaviourally identical across every rendering technology.

Deterministic Rendering

Given identical:

- Component Contracts,
- Accessibility Profiles,
- Runtime state,

Runtime Rendering should produce equivalent presentation.

Perfect pixel matching is unnecessary.

Behavioural equivalence is essential.

Failure Behaviour

Rendering failures should degrade gracefully.

Preferred.

[text]
Advanced Material Fails

↓

Fallback Material

↓

Continue

Avoid.

[text]
Rendering Error

↓

Blank Screen

Presentation quality may reduce.

Behavioural continuity should remain intact.

Plugins

Extensions never interact directly with Runtime Rendering.

Plugins contribute:

- behaviour,
- Expressions,
- information.

The runtime resolves:

- Tiles,
- Components,
- Contracts.

Rendering simply displays them.

Every extension therefore automatically benefits from rendering improvements.

Good Examples

Playback

Playback progresses.

↓

Timeline Contract updates.

↓

Timeline Component renders.

↓

Presentation updates.

Only the affected region redraws.

Reading

Typography profile updates.



Text Components rerender.



Reading continues naturally.

Library

Virtualised collection.



Visible Components rendered.



Behaviour remains identical.

Users never perceive implementation optimisations.

Anti-patterns

Behavioural Rendering

Rendering engine deciding runtime behaviour.

Full Redraw

Entire interface redrawn for local behavioural updates.

Platform Behaviour

Different rendering engines creating different runtime semantics.

Accessibility Drift

Accessible presentation diverging from visual presentation.

Runtime Rendering Model

[mermaid]
flowchart TD

```
ComponentContracts
ComponentContracts --> ChangeDetection
ChangeDetection --> RenderScheduling
RenderScheduling --> Composition
Composition --> Rendering
Rendering --> Presentation
Presentation --> User
```

Rendering faithfully executes runtime decisions.

Behaviour always remains upstream.

Relationship To Future Chapters

The next chapter defines **Component Optimisation**.

Runtime Rendering explains:

How Components become continuously visible.

Component Optimisation explains:

How rendering efficiency improves over time while preserving every behavioural guarantee established by the Mosaic architecture.

Together they complete the implementation pipeline of the Component Library.

Summary

Runtime Rendering is the final implementation stage of the entire Mosaic Design Language.

By the time rendering begins:

- behaviour is solved,
- presentation is solved,
- accessibility is solved.

Rendering simply makes those decisions visible efficiently and consistently.

That deliberate simplicity is one of the defining architectural strengths of Mosaic.

Review Status

Status

Draft

Next File

10-component-optimisation.md

Component Optimisation

Purpose

The Component Library exists to faithfully implement the runtime architecture.

Optimisation should strengthen that implementation.

It should never alter behavioural meaning.

Unlike many UI frameworks, Mosaic treats optimisation as an implementation concern rather than an architectural one.

Performance should improve continuously.

Behaviour should remain identical.

Definition

Within MDS, **Component Optimisation** is defined as:

The implementation techniques used to improve rendering efficiency while preserving deterministic behaviour, presentation fidelity and runtime contracts.

Optimisation changes execution.

It never changes understanding.

Philosophy

Optimisation should answer one question.

"Can we render the same behaviour more efficiently?"

Not:

"Can we simplify behaviour to improve performance?"

Behaviour always possesses higher authority than optimisation.

Optimisation Principles

Every optimisation should satisfy the following principles.

- deterministic
- behaviour preserving
- presentation preserving
- measurable
- reversible

- platform independent

Optimisation should never introduce behavioural side effects.

Behaviour Before Performance

Preferred.

[text]
Same Behaviour

↓

Better Performance

Avoid.

[text]
Simpler Behaviour

↓

Better Performance

Performance improvements should never weaken runtime understanding.

Optimisation Targets

Typical optimisation areas include:

- rendering
- layout
- compositing
- virtualisation
- memory usage
- GPU utilisation
- image decoding
- animation scheduling

Behavioural systems remain outside the scope of optimisation.

Component Reuse

Components should be reused whenever practical.

Examples.

- virtualised collections
- scrolling shelves
- infinite feeds

- recommendation lists

Reuse reduces allocation.

It does not alter behavioural identity.

Virtualisation

Large collections should virtualise presentation.

Example.

[text]
1000 Tiles

↓

Visible Components

↓

Render

Tiles continue existing behaviourally.

Only implementation becomes selective.

Partial Updates

Only affected Components should update.

Preferred.

[text]
Timeline

↓

Timeline Component

↓

Render

Avoid.

[text]
Timeline

↓

Entire Hero Tree

↓

Render

Behavioural locality should naturally produce rendering locality.

Contract Equality

Future implementations should compare Component Contracts before updating.

Conceptually.

[text]
Old Contract

↓

Compare

↓

Changed?

↓

Update

Equivalent Contracts should not trigger unnecessary rendering.

Image Optimisation

Media Components may optimise:

- decoding
- caching
- resolution selection
- progressive loading

These optimisations should remain invisible to users.

Behaviour should remain unchanged.

Material Optimisation

Material rendering may optimise:

- shader reuse
- cached refraction
- atmosphere fields
- layer compositing

Material identity must remain unchanged.

Only implementation improves.

Typography Optimisation

Typography rendering may optimise:

- glyph caching
- paragraph layout
- shaping reuse
- font loading

Editorial hierarchy should remain identical.

Optimisation should never influence readability.

Motion Optimisation

Motion may optimise:

- interpolation
- frame scheduling
- batching
- animation reuse

Motion sequencing must remain behaviourally identical.

Accessibility Optimisation

Accessibility should never be disabled for performance.

Instead optimise:

- semantic tree generation
- focus updates
- accessibility caching

Accessibility correctness always has higher priority than rendering speed.

Memory Optimisation

Future implementations should minimise:

- duplicate Components
- duplicated textures
- repeated allocations
- unnecessary object graphs

Memory optimisation should remain behaviourally transparent.

GPU Optimisation

GPU acceleration may improve:

- Acrylic
- Refraction
- Atmosphere
- compositing
- transitions

GPU techniques remain implementation details.

They should never become architectural concepts.

Lazy Rendering

Presentation may be deferred until required.

Examples.

- off-screen collections
- hidden overlays
- collapsed groups

Deferred rendering should never delay behavioural updates.

The Runtime World always remains current.

Platform Optimisation

Each platform may optimise differently.

Flutter.

↓

Widget reuse.

Web.

↓

DOM reuse.

SwiftUI.

↓

View identity.

Compose.

↓

Recomposition.

Implementation differs.

Behaviour remains identical.

Performance Profiles

Future implementations may expose conceptual performance profiles.

Examples.

[text]
Maximum Quality

↓

Highest Fidelity

[text]
Balanced

↓

Adaptive Optimisation

[text]
Efficiency

↓

Simplified Rendering

Performance Profiles should never alter:

- behaviour
- hierarchy
- interaction
- accessibility

Only rendering fidelity changes.

Runtime Metrics

Future implementations should collect metrics such as:

- frame time
- render cost
- contract updates
- virtualisation efficiency
- cache hit rate
- memory usage

Metrics exist to improve implementation.

They should never influence runtime behaviour directly.

Failure Behaviour

Optimisation failures should degrade gracefully.

Preferred.

```
[text]
GPU Optimisation Disabled
```

↓

CPU Rendering

↓

Continue

Avoid.

```
[text]
Optimisation Failed
```

↓

Behaviour Changes

Optimisation should always remain optional.

Behaviour should never depend upon it.

Plugins

Extensions never participate in optimisation.

Plugins contribute:

- behaviour
- information
- Expressions

Platform implementations optimise rendering transparently.

Every extension therefore automatically benefits from future optimisation work.

Good Examples

Playback

Timeline updates.

↓

Single Component updates.

↓

Partial redraw.



Behaviour preserved.

Library

Large collection.



Virtualisation.



Smooth scrolling.



Identical behavioural understanding.

Hero

Cached Material.



Reduced GPU work.



Identical presentation.

Users perceive no difference.

Anti-patterns

Behaviour Optimisation

Changing runtime behaviour for performance.

Accessibility Reduction

Removing accessibility to improve rendering speed.

Platform Behaviour

Different optimisation strategies producing different behavioural outcomes.

Smart Components

Components introducing behavioural shortcuts to improve performance.

Component Optimisation Model

[mermaid]
flowchart TD

```
ComponentContracts
ComponentContracts --> OptimisationLayer
OptimisationLayer --> PlatformComponents
PlatformComponents --> Rendering
Rendering --> Presentation
```

Optimisation improves implementation.

It never changes behaviour.

Relationship To Future Chapters

The next chapter defines **Component Library Governance**.

Component Optimisation explains:

How implementation becomes more efficient.

Governance explains:

How the Component Library evolves while preserving the architectural guarantees established throughout the Mosaic Design Language.

Together they complete the long-term implementation strategy of the Component Library.

Summary

Component Optimisation is intentionally invisible.

Users should never notice:

- caching,
- virtualisation,
- GPU acceleration,
- rendering reuse.

They should simply experience a Companion that feels:

- fast,
- responsive,
- effortless.

The implementation may become dramatically more sophisticated over time.

The behavioural experience should remain exactly the same.

Review Status

Status

Draft

Next File

11-governance.md

Component Library Governance

Purpose

The Component Library is the final implementation layer of the Mosaic architecture.

Unlike runtime systems, Components are expected to evolve frequently as:

- UI frameworks mature,
- rendering technologies improve,
- platform capabilities expand.

Despite this implementation flexibility, the behavioural language of Mosaic must remain completely stable.

This chapter defines how the Component Library should evolve while preserving that separation.

Governance Philosophy

Components should evolve continuously.

Behaviour should not.

The objective is not preserving widgets.

It is preserving the architectural guarantee that:

Components render decisions. They never make them.

Every governance decision should reinforce this separation.

Components Are Implementation

Components should always remain implementation artefacts.

Changing a Component should never require changes to:

- Behaviour
- Runtime World
- Composition
- Expressions
- Tiles

If upstream architecture changes because a Component changed...

The architectural boundary has been violated.

Stable Responsibilities

The following concepts should remain highly stable.

- Component Philosophy
- Component Contracts
- Component Lifecycle
- Accessibility Contracts
- Platform-neutral rendering model

These concepts define the public implementation architecture of Mosaic.

Evolvable Responsibilities

The following may evolve continuously.

- framework integrations
- rendering optimisations
- GPU techniques
- virtualisation
- memory management
- platform widgets
- graphics APIs

Implementation should improve.

Behaviour should remain identical.

Component Ownership

Responsibilities are intentionally separated.

Layer	Owner
Component Philosophy	Design Systems
Component Contracts	Runtime Architecture
Platform Components	Client Platform Teams
Rendering	Platform Rendering Teams
Performance Optimisation	Platform Runtime Teams

Ownership protects architectural boundaries while allowing implementation teams to innovate independently.

Introducing New Components

Before creating a new Component ask:

Question One

Can an existing Component already render this Contract?

Question Two

Is this a genuinely new implementation primitive...
or merely another visual variation?

Question Three

Could Component Composition solve this instead?

Question Four

Will this Component remain meaningful after the current UI framework becomes obsolete?

Question Five

Would another contributor naturally discover this Component?

If uncertainty remains...

Refine the proposal before implementation.

Component Drift

Component Drift occurs when:

- Components begin solving behaviour,
- framework-specific Components appear,
- Components reinterpret runtime Contracts,
- implementation begins influencing architecture,
- rendering logic leaks into runtime.

Component Drift weakens one of Mosaic's strongest architectural guarantees.

It should therefore be treated as architectural debt.

Component Debt

Examples include:

- duplicate implementation primitives,
- stateful Components,
- framework-specific abstractions,

- behaviour hidden inside Components,
- undocumented rendering exceptions.

Component Debt should be reduced continuously.

Implementation should become simpler over time.

Not more complicated.

Contract Governance

Component Contracts represent one of the strongest architectural boundaries within Mosaic.

Components should consume Contracts exactly as provided.

They should never:

- reinterpret,
- modify,
- extend,
- override

runtime Contracts independently.

Platform Governance

Every platform should implement identical behavioural contracts.

Flutter.

↓

Same behaviour.

Web.

↓

Same behaviour.

SwiftUI.

↓

Same behaviour.

Compose.

↓

Same behaviour.

Rendering quality may differ.

Behavioural meaning must not.

Accessibility Governance

Accessibility always possesses higher authority than rendering quality.

No Component proposal should weaken:

- accessibility,
- readability,
- interaction,
- semantic correctness.

Accessibility should remain entirely contract driven.

Performance Governance

Performance optimisation should never alter:

- runtime hierarchy,
- Materials,
- Typography,
- Motion,
- interaction,
- behavioural sequencing.

Optimisation improves implementation only.

Plugin Governance

Extensions must never provide:

- Components,
- rendering,
- framework integrations,
- platform widgets.

Plugins contribute:

- behaviour,
- Expressions,
- information.

The platform owns implementation.

This guarantees one coherent rendering language across the ecosystem.

Review Questions

Every Component proposal should answer:

- Does this render an existing Contract?
- Does it remain behaviourally passive?
- Could an existing Component already perform this work?
- Would this remain useful after a framework migration?
- Does it preserve platform independence?
- Is this solving implementation rather than architecture?

If the proposal exists primarily because a framework encourages it...

It should be reconsidered.

Frameworks are temporary.

Architecture is not.

Validation

Future tooling should automatically validate:

- Component Contract compliance
- platform parity
- accessibility correctness
- rendering determinism
- Component reuse
- absence of behavioural logic

Validation should reinforce architectural review.

It should never replace thoughtful implementation design.

Governance Workflow

[mermaid]
flowchart TD

```
Proposal
Proposal --> ExistingComponentReview
ExistingComponentReview --> Reuse["Reuse?"]
Reuse["Reuse?"] -->|Yes| ExistingComponent
Reuse["Reuse?"] -->|No| ArchitectureReview
ArchitectureReview
ArchitectureReview --> AccessibilityReview
AccessibilityReview --> PlatformReview
PlatformReview --> Approval
Approval --> Implementation
```

Reuse should remain the preferred outcome.

The implementation vocabulary should remain intentionally small.

Success Criteria

The Component Library succeeds when:

- Components remain behaviourally passive,
- runtime Contracts remain authoritative,
- rendering frameworks remain replaceable,
- contributors naturally think in Tiles rather than Components,
- accessibility remains automatic,
- platform implementations remain behaviourally identical.

Users should never perceive Components.

They should simply experience a platform that behaves consistently everywhere.

Architectural Decisions

ADR	Decision
ADR-177	Components are implementation artefacts rather than behavioural objects.
ADR-178	Component Contracts are immutable runtime boundaries.
ADR-179	Platform implementations never redefine runtime behaviour.
ADR-180	Accessibility is contract-driven rather than component-driven.
ADR-181	Extensions never implement Components directly.

Review Status

Status

Draft

Next File

12-adr s .md

Architectural Decision Records

Purpose

The Architectural Decision Records (ADRs) contained within MDS-008 preserve the architectural reasoning behind the Mosaic Component Library.

Every previous specification established:

- Behaviour
- Runtime World
- Composition
- Expressions
- Tiles

MDS-008 establishes how those concepts become concrete platform implementations.

These ADRs explain why Mosaic deliberately treats Components as implementation artefacts rather than behavioural objects.

Future contributors should understand these decisions before modifying the Component Library.

ADR Format

Every Mosaic ADR follows the standard structure.

[text]

ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications

Each ADR documents one architectural decision.

ADR-182

Title

Components Render Rather Than Reason

Status

Accepted

Context

Traditional UI frameworks frequently allow Components to own behaviour and application state.
Founder workshops consistently reinforced that behaviour should remain external to implementation.

Decision

Components become passive renderers of resolved runtime Contracts.

Consequences

Behaviour remains framework independent while rendering technologies become replaceable.

ADR-183

Title

Introduce Immutable Component Contracts

Status

Accepted

Context

Allowing Components to reinterpret runtime information fragments behavioural consistency.

Decision

Every Component consumes immutable runtime Contracts.
Components never modify them.

Consequences

Presentation becomes deterministic across every Mosaic client.

ADR-184

Title

Separate Component Identity From Tile Identity

Status

Accepted

Context

Behavioural presentation and rendering implementation evolve at different rates.

Decision

Tiles preserve behavioural identity.

Components preserve implementation identity.

Consequences

Rendering frameworks may evolve independently from runtime architecture.

ADR-185

Title

Accessibility Is Resolved Before Rendering

Status

Accepted

Context

Platform-specific accessibility implementations frequently become inconsistent.

Decision

Accessibility becomes part of Component Contracts rather than Component logic.

Consequences

Every platform receives identical behavioural accessibility semantics.

ADR-186

Title

Platform Components Never Redefine Behaviour

Status

Accepted

Context

Different UI frameworks naturally encourage different implementation patterns.

Decision

Platform Components faithfully implement runtime Contracts without reinterpretation.

Consequences

Flutter, Web, SwiftUI and Compose all communicate identical behavioural understanding.

ADR-187

Title

Optimisation Must Preserve Behaviour

Status

Accepted

Context

Rendering optimisations frequently simplify runtime behaviour for performance.

Decision

Performance improvements must preserve behavioural correctness completely.

Consequences

Optimisation becomes transparent to users.

ADR-188

Title

Component Lifecycle Is Independent From Tile Lifecycle

Status

Accepted

Context

Behavioural continuity should survive implementation changes.

Decision

Components may be recreated, pooled or virtualised independently from behavioural Tiles.

Consequences

Rendering flexibility increases without weakening user understanding.

ADR-189

Title

Rendering Is The Final Architectural Layer

Status

Accepted

Context

Many frameworks blur the distinction between runtime architecture and rendering.

Decision

Rendering becomes the final implementation stage only.

Consequences

Every upstream architectural layer remains independent from graphics technology.

ADR-190

Title

Extensions Never Implement Components

Status

Accepted

Context

Plugin-owned rendering fragments presentation quality and behavioural consistency.

Decision

Extensions contribute runtime behaviour only.

The platform owns Components completely.

Consequences

Community extensions automatically inherit future rendering improvements.

ADR Relationships

[mermaid]
flowchart TD

ADR182["Passive Components"]

ADR183["Immutable Contracts"]

ADR184["Identity Separation"]

ADR185["Accessibility"]

ADR186["Platform Fidelity"]

ADR187["Optimisation"]

ADR188["Lifecycle"]

ADR189["Rendering"]

ADR190["Extensions"]

ADR182 --> ADR183

ADR183 --> ADR184

ADR184 --> ADR188

ADR183 --> ADR185

ADR183 --> ADR186

ADR186 --> ADR189

ADR187 --> ADR189

ADR189 --> ADR190

Together these decisions establish the Component Library as a thin implementation layer that faithfully renders the runtime architecture without altering it.

Future ADRs

Future Component Library ADRs are expected to formalise:

- GPU-driven Component Rendering
- Streaming Component Trees
- Server-assisted Rendering
- Spatial UI Components
- Adaptive Rendering Pipelines
- AI-assisted Rendering Optimisation
- Hardware Capability Profiles
- Runtime Rendering Personas

These intentionally remain outside the scope of MDS-008 Version 0.1.

ADR Governance

Component Library ADRs should change only when:

- rendering architecture fundamentally evolves,

- accessibility standards require refinement,
- platform implementation strategies change,
- the Mosaic Design Language itself evolves.

Framework trends alone should never justify architectural changes.

Components should remain behaviourally passive regardless of future implementation technology.

Summary

The ADRs contained within MDS-008 define the implementation identity of Mosaic.

Behaviour belongs to the Runtime World.

Presentation belongs to Tiles.

Implementation belongs to Components.

Rendering belongs to platforms.

Maintaining these boundaries allows Mosaic to continuously adopt new UI technologies without ever compromising its behavioural architecture.

Review Status

Status

Draft

Next File

13-contributor-guidance.md

Contributor Guidance

Purpose

The Component Library is the only part of Mosaic most frontend engineers will interact with every day.

It is therefore the easiest place for architectural boundaries to erode.

This guidance exists to ensure contributors continue thinking in terms of:

- Behaviour,
- Expressions,
- Tiles,
- Contracts,

rather than:

- widgets,
- callbacks,
- screens,
- application state.

When contributors naturally think in runtime architecture before implementation, the Component Library has achieved its purpose.

Think In Contracts

Never begin with:

"How should this widget behave?"

Instead ask:

"Which Component Contract am I rendering?"

Good.

[text]
Component Contract

↓

Rendering

Poor.

[text]
widget

↓

Behaviour

Components should consume behaviour.

Never invent it.

Think In Tiles

Every Component exists because a Tile exists.

Before implementing anything ask:

"Which Tile am I rendering?"

If the answer is unclear...

The implementation should probably not begin yet.

Components Never Decide

Components should never determine:

- hierarchy,
- colour,
- typography,
- motion,
- interaction,
- behaviour.

Those decisions already exist.

If a Component needs to decide...

The architecture has already drifted.

Preserve Architectural Boundaries

Preferred.

[text]
Behaviour

↓

Expression

↓

Tile

↓

Component

↓

Rendering

Avoid.

[text]
Component

↓

Behaviour

↓

Rendering

Dependencies should always flow downward.

Never upward.

Respect Contracts

Component Contracts should be treated as immutable.

Never:

- modify,
- reinterpret,
- extend,
- override

a Contract inside a Component.

If the Contract is insufficient...

Improve the runtime architecture.

Not the Component.

Stay Stateless

Whenever practical...

Components should remain stateless.

Preferred.

[text]
Contract

↓

Render

Avoid.

[text]
Component State

↓

Behaviour

↓

Render

State belongs to the Runtime World.

Components simply display it.

Let Runtime Own Updates

Components should respond to:

[text]
New Contract

They should never poll:

- application state,
- services,
- runtime systems.

The Composition Engine already solved the user's World.

Trust it.

Reuse Components

Before creating a new Component ask:

Can an existing implementation primitive already render this Contract?

Most new implementation requests should improve:

- Component Composition,
- Component Contracts,

rather than expanding the Component vocabulary.

Respect Accessibility

Accessibility should always originate from Accessibility Contracts.

Components should never invent:

- semantics,
- hierarchy,
- reading order,
- behavioural actions.

Platform APIs implement accessibility.

Runtime defines it.

Respect Platform Independence

Before implementing any Component ask:

Would another platform be able to render this Contract using an equivalent implementation?

If the answer is no...

The Component is probably too platform-specific.

Performance Comes Last

Optimise only after correctness.

Preferred.

[text]
Correct

↓

Optimised

Avoid.

[text]
Optimised

↓

Different Behaviour

Behavioural correctness always has higher priority than rendering performance.

Plugins Never Render

Extensions contribute:

- behaviour,
- Expressions,
- information.

They never contribute:

- Components,
- rendering,
- UI frameworks.

Every extension should inherit presentation automatically.

Common Mistakes

Avoid the following.

Smart Components

Components performing runtime reasoning.

Mutable Contracts

Components changing runtime Contracts.

Platform Logic

Platform-specific Components introducing behavioural differences.

Widget Thinking

Naming Components after application features.

Behavioural Rendering

Rendering frameworks influencing runtime behaviour.

Accessibility Logic

Components independently implementing accessibility policy.

Component Review Questions

Before implementing any Component ask:

- Which Tile does this render?
- Which Contract does it consume?
- Does it remain behaviourally passive?
- Could an existing Component already perform this work?
- Would this implementation work on every Mosaic platform?
- Does it preserve architectural boundaries?

If uncertainty remains...

Return to the Tile before writing implementation.

Component Checklist

Every Component implementation should satisfy the following.

- ☐ Consumes immutable Component Contracts.
- ☐ Contains no behavioural logic.
- ☐ Remains platform independent.

- [] Preserves accessibility.
- [] Preserves Material behaviour.
- [] Preserves Typography behaviour.
- [] Preserves Motion behaviour.
- [] Remains stateless wherever practical.

Final Guidance

The Component Library should eventually disappear from architectural discussion.

Contributors should stop asking:

"Which widget should do this?"

and instinctively begin asking:

"Which resolved behaviour am I implementing?"

When every contributor naturally thinks this way, Components become exactly what they should be:

Simple.

Predictable.

Replaceable.

The runtime architecture remains the intelligence of Mosaic.

The Component Library simply gives that intelligence a visible form.

Review Status

Status

Draft

Next File

glossary.md

Glossary

Purpose

This glossary defines the terminology introduced by **MDS-008 — Component Library**.

Unlike previous glossaries, these definitions describe the final implementation layer of the Mosaic architecture.

These terms intentionally remain independent from:

- UI frameworks,
- rendering engines,
- graphics APIs,
- programming languages.

Future specifications should reuse these definitions consistently.

A

Accessibility Contract

An immutable runtime contract describing the accessibility behaviour a Component must implement.

Accessibility Contracts include:

- semantic roles,
- accessible names,
- focus behaviour,
- interaction actions,
- motion preferences.

Accessibility Contracts are produced by the runtime.

Components consume them.

C

Component

A platform-specific implementation primitive responsible for rendering a resolved Tile.

Components are implementation artefacts.

They do not own:

- behaviour,
- hierarchy,

- Materials,
- Typography,
- Motion.

Component Contract

The immutable runtime description consumed by a Component.

Component Contracts contain:

- Material Profile,
- Typography Profile,
- Motion Profile,
- Interaction Profile,
- Accessibility Profile.

Components render Contracts.

They never modify them.

Component Composition

The deterministic assembly of Components required to render one resolved Tile.

Component Composition concerns implementation only.

Component Lifecycle

The implementation lifecycle through which Components are:

- created,
- bound,
- rendered,
- updated,
- detached,
- released.

This lifecycle is intentionally separate from Tile lifecycle.

M

Material Profile

The resolved Material behaviour supplied to Components through Component Contracts.

Material Profiles describe implementation behaviour.

Components execute them.

P

Platform Component

A framework-specific implementation of a Mosaic Component Contract.

Examples include:

- Flutter Component,
- React Component,
- SwiftUI Component,
- Compose Component.

Platform Components preserve identical behavioural presentation.

Presentation

The visible result produced by rendering Components.

Presentation is the final architectural output of the Mosaic Design Language.

R

Rendering

The implementation process that transforms Platform Components into visible presentation.

Rendering is behaviourally passive.

It never determines runtime understanding.

Runtime Rendering

The deterministic execution of Component Contracts into continuously updated presentation.

Runtime Rendering performs:

- scheduling,
- compositing,
- drawing,
- accessibility integration.

It does not perform behavioural reasoning.

T

Tile

A behavioural presentation primitive defined by the Tile Framework.

Components render Tiles indirectly through Component Contracts.

Typography Profile

The resolved typography behaviour supplied to Text Components.

Typography Profiles determine:

- hierarchy,
- scale,
- rhythm,
- accessibility.

Components simply render them.

V

Virtualisation

An implementation optimisation allowing Components to be created only when required.

Virtualisation affects implementation only.

Behaviour remains identical.

Cross References

Specification	Primary Concepts
MDL-001 Vision	Companion
MDL-002 Principles	Behaviour Before Interface
MDL-003 Mental Model	World
MDL-004 Interaction Model	Behaviour
MDL-005 Composition Model	Hierarchy
MDS-006 Composition Engine	Expressions, Presentation Model
MDS-007 Tile Framework	Tiles, Runtime Tile Resolution

Terminology Rules

Future contributors should:

- describe Tiles before Components,
- describe Contracts before rendering,
- distinguish implementation from behaviour,
- distinguish rendering from runtime architecture,
- avoid framework-specific terminology within architectural documentation.

Component terminology should remain independent from implementation technologies.

Review Status

Status

Draft

Next File

references.md

References

Purpose

This document records the architectural influences and conceptual foundations that informed **MDS-008 — Component Library**.

Unlike implementation documentation, these references explain *why* the Component Library exists rather than documenting specific frameworks or rendering APIs.

The Component Library intentionally combines ideas from:

- behavioural architecture,
- declarative rendering,
- runtime contracts,
- accessibility,
- platform abstraction,
- deterministic execution,

into one implementation layer that remains deliberately thin.

Every architectural decision before this specification already determined behaviour.

The Component Library simply makes those decisions visible.

Reading Order

Contributors should approach references in the following order.

1. MDL Specifications 2. Design Token Architecture 3. Colour System 4. Material System 5. Typography System 6. Motion System 7. Composition Engine 8. Tile Framework 9. Component Library 10. Platform Implementations

The Mosaic Design Language remains the architectural authority.

Rendering technologies merely implement it.

Internal References

MDL-001 — Vision

Provides:

- Companion philosophy
- Entertainment-first thinking
- Calm interaction

Components should remain invisible beneath the Companion.

Users should remember the experience.

Never the implementation.

MDL-002 — Principles

Provides:

- Behaviour Before Interface
- Content Leads
- Calm Interfaces
- Every Feature Earns Its Place

Components exist only because behaviour has already been solved.

Implementation should never redefine those principles.

MDL-003 — Mental Model

Provides:

- World
- Focus
- Context
- Relationships

Components never reason about the World.

They simply render it.

MDL-004 — Interaction Model

Provides:

- Behaviour
- Continuity
- Runtime evolution

Components execute interaction.

They never determine interaction.

MDL-005 — Composition Model

Provides:

- Hero
- Hierarchy
- Priority

- Expressions

Component trees should always reflect solved Composition rather than invent their own hierarchy.

MDS-001 — Design Token Architecture

Provides:

- Runtime Tokens
- Resolution
- Semantic contracts

Component Contracts continue the runtime resolution architecture established by MDS-001.

MDS-002 — Colour System

Provides:

- Runtime Atmosphere
- Material colour
- Semantic colour

Components consume resolved colour behaviour.

They never determine colour independently.

MDS-003 — Material System

Provides:

- Hero Material
- Acrylic
- Runtime Material Resolution

Components implement Materials.

They never reinterpret them.

MDS-004 — Typography System

Provides:

- Editorial Hierarchy
- Typography Resolution
- Accessibility

Text Components consume resolved Typography Profiles.

Editorial decisions always remain upstream.

MDS-005 — Motion System

Provides:

- Behavioural Motion
- Motion Resolution
- Accessibility Motion

Components execute Motion Profiles.

Behavioural sequencing remains runtime owned.

MDS-006 — Composition Engine

Provides:

- Runtime World
- Expressions
- Runtime Hierarchy
- Presentation Models

Components never consume runtime behaviour directly.

They consume only resolved presentation.

MDS-007 — Tile Framework

Provides:

- Tile Philosophy
- Runtime Tile Resolution
- Tile Contracts

The Component Library exists solely because Tiles require physical implementation.

Without Tiles, Components possess no architectural meaning.

Architectural Principle

The Component Library intentionally concludes the architectural pipeline.

Conceptually.

[text]
Behaviour

↓

Composition

↓

Expressions

↓

Tiles

↓

Components

↓

Rendering

↓

User

Everything upstream determines meaning.

Everything downstream implements meaning.

Declarative Rendering

The Component Library is heavily influenced by declarative rendering.

Components describe:

What should be rendered.

Never:

What should happen.

This distinction significantly simplifies long-term maintenance.

Deterministic Systems

The Component Library assumes deterministic rendering.

Identical:

- Component Contracts,
- runtime profiles,
- accessibility,

should always produce equivalent presentation.

This assumption enables:

- replay,
- testing,
- visual regression,
- platform parity.

Accessibility

Accessibility is treated as an architectural requirement rather than a platform feature.

The Component Library assumes:

[text]
Runtime

↓

Accessibility Contracts

↓

Components

↓

Accessible Presentation

Accessibility therefore remains behaviourally identical across every supported client.

Platform Independence

The Component Library intentionally separates:

[text]
Component Contracts

↓

Platform Components

↓

Rendering

Future rendering technologies should require only:

- new platform Components,
- new rendering adapters.

Behavioural architecture remains unchanged.

Rendering Technologies

Rendering technologies are intentionally excluded from architectural decision-making.

Examples include:

- Flutter Impeller
- Skia
- Metal
- Vulkan
- WebGPU

- DirectX
- HTML Canvas

These technologies implement Components.

They never define Components.

Performance

Performance improvements should remain implementation concerns.

Examples include:

- virtualisation,
- batching,
- layer caching,
- shader optimisation,
- recomposition reduction.

Optimisation should never modify:

- hierarchy,
- Materials,
- Typography,
- Motion,
- interaction,
- accessibility.

Behaviour remains the architectural authority.

Mosaic-Specific Influences

The Component Library emerged directly from founder exploration.

Major architectural discoveries included:

- Components should become almost behaviourless.
- Runtime Contracts should eliminate component decision-making.
- Tiles provide the correct abstraction between behaviour and implementation.
- Rendering should become entirely replaceable.
- Accessibility should be resolved before Components exist.

Together these discoveries complete the architectural separation that defines Mosaic.

Relationship To The Companion

Components represent the visible body of the Companion.

Conceptually.

[text]
World

↓

Behaviour

↓

Expressions

↓

Tiles

↓

Components

↓

Presentation

↓

User

The Companion understands first.

Components merely give that understanding physical form.

This ordering should remain fundamental throughout the lifetime of Mosaic.

Normative References

Required reading before contributing to MDS-008.

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography System
- MDS-005 Motion System
- MDS-006 Composition Engine

- MDS-007 Tile Framework

Together these specifications define the complete architectural foundation of the Component Library.

Informative References

Future implementation guides may define:

- Flutter implementation standards
- Web implementation standards
- SwiftUI implementation standards
- Compose implementation standards
- Rendering optimisation guides
- Platform accessibility guides

These documents should consume the Component Library.

They should never redefine it.

Living Document

This reference list should remain intentionally concise.

References should only be introduced when they materially influence:

- implementation architecture,
- rendering,
- accessibility,
- runtime contracts,
- platform independence.

The objective is to preserve architectural reasoning rather than catalogue framework documentation.

Completion

This concludes **MDS-008 — Component Library**.

It also completes the **Mosaic Design System (MDS)**.

Together, the MDL and MDS define a complete architecture spanning:

- **Vision**
- **Principles**
- **Mental Models**
- **Interaction**
- **Composition**

- **Design Tokens**
- **Colour**
- **Materials**
- **Typography**
- **Motion**
- **Runtime Composition**
- **Presentation**
- **Implementation**

From this point onwards, future documentation should transition from design architecture into product and engineering specifications, such as:

- Mosaic Runtime Architecture
- Extension SDK
- Storage Architecture
- Media Pipeline
- Networking
- Plugin Runtime
- API Specifications
- Client Implementation Guides

At this stage, the behavioural language of Mosaic is fully defined.

Everything that follows is engineering.